

IMPLEMENTACIÓN DE OBSERVABILIDAD, TRAZABILIDAD Y SEGURIDAD – NUTRIVET AI V3.0

Asignatura: Ingeniería de Soluciones con IA (ISY0101)

Evaluación: Evaluación Parcial N°3

Integrantes: Gabriel Cares y Abner Gutierrez

1. Introducción y Arquitectura del Agente

El presente informe técnico detalla la incorporación de capacidades de observabilidad, auditoría transaccional y mitigación de riesgos de seguridad sobre el Agente Clínico Autónomo **NutriVet AI v3.0**. El sistema, estructurado sobre una arquitectura híbrida de Generación Aumentada por Recuperación (RAG), procesa consultas veterinarias cruzando datos estructurados (datos_razas.json) con las Guías Técnicas de Evaluación Nutricional de la WSAVA (2011). En esta iteración, el núcleo ha sido robustecido para responder a demandas operacionales de un entorno en producción, permitiendo visibilizar el rendimiento del LLM (gpt-4o-mini) mediante métricas en tiempo real.

2. Diseño e Implementación de Métricas de Observabilidad (IL3.1 e IL3.5)

Para cuantificar de manera empírica el desempeño de la solución ante la variabilidad de los inputs, se diseñó e integró un set de métricas clave (KPIs) computadas directamente en el ciclo de ejecución:

- **Latencia Transaccional (s):** Se implementó una medición precisa utilizando la librería “Time” para registrar la estampa de tiempo inmediatamente antes de la inferencia y compararla con el tiempo de respuesta final del modelo de lenguaje.
- **Frecuencia de Éxitos y Errores Críticos:** Se estructuró un bloque de captura de excepciones (try-except) que clasifica de forma determinista las respuestas exitosas frente a fallas de conexión del endpoint de OpenAI/GitHub o fallos en la búsqueda semántica de la base vectorial ChromaDB.
- **Dashboard Visual en Consola/UI:** El sistema renderiza un cuadro dinámico superior que actualiza el promedio móvil de latencia y la tasa de error por sesión de manera inmediata tras cada consulta.

EXPLORER

PROYECTO IA PARA MASCOTAS

chroma_db

.env

datos_razas.json

Motor_IA.py

nutrivet_agente.log

Requerimientos.txt

WSAVA-Nutrition-Assessm...

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Bulldog Francés	Perro	✓ Disponible
Bulldog Inglés	Perro	✓ Disponible
Labrador Retriever	Perro	✓ Disponible
Chihuahua	Perro	✓ Disponible
Dachshund (Teckel)	Perro	✓ Disponible
Gato Persa	Gato	✓ Disponible
Maine Coon	Gato	✓ Disponible
Siamés	Gato	✓ Disponible
Esfinge (Sphinx)	Gato	✓ Disponible
British Shorthair	Gato	✓ Disponible

DASHBOARD DE OBSERVABILIDAD EN VIVO (Métricas de Producción):

-> Consultas Totales: 4 | Exitosas: 2

-> Filtros de Seguridad Activados: 1 | Errores Críticos: 1

-> Latencia Promedio: 16.3 segundos

ÚLTIMO DIAGNÓSTICO SUGERIDO:

El Golden Retriever presenta los siguientes riesgos genéticos:

- Displasia de cadera y codo.

- Tendencia a la obesidad.

- Mayor riesgo de desarrollo de tumores/cáncer.

Es importante tener en cuenta estos riesgos para implementar medidas de salud preventiva y nutricional adecuadas.

OUTLINE

TIMELINE

Ingrese síntoma o consulta (o 'salir'):

3. Estrategia de Trazabilidad y Análisis de Registros (IL3.2 e IL3.4)

La trazabilidad se resolvió mediante la suite nativa “logging” de Python, configurando un manejador de archivos (FileHandler) persistente que escribe de manera asíncrona en el fichero local “nutrivet_agente.log”.

- **Estructura del Registro:** Cada línea del log estandariza el formato [AAAA-MM-DD HH:MM:SS] [NIVEL] Mensaje explicativo.
- **Análisis ante Puntos de Falla:** Los niveles de severidad (INFO, WARNING, ERROR) permiten aislar fallas de infraestructura (pistas de errores del RAG) de eventos operacionales regulares.

```
*nutrivet_agente: Bloc de notas
Archivo Edición Formato Ver Ayuda
2026-07-09 21:49:25,090 [INFO] Cargando repositorio vectorial ChromaDB desde persistencia local.
2026-07-09 21:50:11,185 [INFO] Sesión clínica finalizada por el usuario voluntariamente.
2026-07-09 21:50:24,225 [INFO] Cargando repositorio vectorial ChromaDB desde persistencia local.
2026-07-09 21:50:56,151 [INFO] Transacción iniciada - Input de usuario: 'Si tengo a mi chiguagua enfermo que hago'
2026-07-09 21:50:58,122 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 21:51:05,607 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/chat/completions "HTTP/1.1 200 OK"
2026-07-09 21:51:05,635 [INFO] Transacción completada exitosamente. Latencia: 9.48s
2026-07-09 21:51:29,904 [INFO] Transacción iniciada - Input de usuario: 'Gracias'
2026-07-09 21:51:32,107 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 21:51:34,014 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/chat/completions "HTTP/1.1 200 OK"
2026-07-09 21:51:34,017 [INFO] Transacción completada exitosamente. Latencia: 4.11s
2026-07-09 21:52:42,193 [INFO] Cargando repositorio vectorial ChromaDB desde persistencia local.
2026-07-09 21:53:23,639 [INFO] Transacción iniciada - Input de usuario: 'Mi perro esta con vomito y esta temblando, es una mezcla de chiguagua con shitsu'
2026-07-09 21:53:25,226 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 21:53:33,194 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/chat/completions "HTTP/1.1 200 OK"
2026-07-09 21:53:33,210 [INFO] Transacción completada exitosamente. Latencia: 9.57s
2026-07-09 22:41:57,847 [INFO] Cargando repositorio vectorial ChromaDB desde persistencia local.
2026-07-09 22:42:47,446 [INFO] Transacción iniciada - Input de usuario: '¿Qué riesgos genéticos tiene el Golden Retriever?'
2026-07-09 22:43:05,371 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 22:43:20,496 [INFO] Retrying request to /chat/completions in 0.411425 seconds
2026-07-09 22:43:32,938 [INFO] Retrying request to /chat/completions in 0.796891 seconds
2026-07-09 22:43:41,877 [ERROR] Falla crítica en respuesta del Agente: Connection error. | Latencia registrada: 54.43s
2026-07-09 22:44:23,532 [INFO] Transacción iniciada - Input de usuario: '¿Qué riesgos genéticos tiene el Golden Retriever?'
2026-07-09 22:44:24,866 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 22:44:27,743 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/chat/completions "HTTP/1.1 200 OK"
2026-07-09 22:44:27,763 [INFO] Transacción completada exitosamente. Latencia: 4.23s
2026-07-09 22:44:43,029 [INFO] Transacción iniciada - Input de usuario: 'Me duele la cabeza, ¿puedo tomar paracetamol?'
2026-07-09 22:44:43,810 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 22:44:45,814 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/chat/completions "HTTP/1.1 200 OK"
2026-07-09 22:44:45,818 [WARNING] Protocolo de seguridad activado: Intento de consulta no permitida / Inyección bloqueada de manera segura. Latencia: 2.79s
2026-07-09 22:54:40,508 [INFO] Transacción iniciada - Input de usuario: '¿Qué riesgos genéticos tiene el Golden Retriever?'
2026-07-09 22:54:42,945 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/embeddings "HTTP/1.1 200 OK"
2026-07-09 22:54:44,270 [INFO] HTTP Request: POST https://models.inference.ai.azure.com/chat/completions "HTTP/1.1 200 OK"
2026-07-09 22:54:44,274 [INFO] Transacción completada exitosamente. Latencia: 3.77s
```

4. Protocolos de Seguridad y Uso Responsable (IL3.3)

Con el propósito de cumplir con los lineamientos éticos, normativos y de privacidad indispensables en un software de salud en producción, se inyectó un **Meta-Prompt System blindado** con dos capas defensivas:

1. **Filtro Temático Veterinario Estricto:** Si el usuario desvía la consulta hacia enfermedades humanas, dosis de fármacos para personas (ej. paracetamol) o dietas de adultos, el sistema intercepta semánticamente el prompt y gatilla una respuesta estandarizada de rechazo absoluto, protegiendo la responsabilidad civil del sistema.
2. **Mitigación de Prompt Injection (Ataques Adversarios):** El modelo tiene instrucciones explícitas de bajo nivel (baja temperatura a **0.1** para asegurar predictibilidad) para ignorar comandos maliciosos que intenten sobrescribir las reglas base o extraer el prompt original del sistema.

```
📊 DASHBOARD DE OBSERVABILIDAD EN VIVO (Métricas de Producción):
-> Consultas Totales: 5 | Exitosas: 2
-> Filtros de Seguridad Activados: 2 | Errores Críticos: 1
-> Latencia Promedio: 13.79 segundos
-----

📄 ÚLTIMO DIAGNÓSTICO SUGERIDO:
Lo siento, como agente clínico veterinario, estoy estrictamente programado para responder consultas relacionadas con la salud y nutrición de mascotas. No p
uedo procesar solicitudes humanas.
-----

🔍 Ingrese síntoma o consulta (o 'salir'):
```

5. Propuesta de Optimización, Escalabilidad y Rediseño (IL3.4)

Basado en el análisis de las métricas de latencia recopiladas durante las pruebas, se proponen dos mejoras de arquitectura esenciales para asegurar la sostenibilidad y la escalabilidad del agente de IA:

1. **Capa de Caché Semántica (Semantic Caching):** Implementar herramientas como *GPTCache*. Almacenar en una base de datos clave-valor las consultas recurrentes respecto a razas típicas (ej. dietas de Golden Retriever). Si un nuevo cliente consulta algo similar, el sistema responderá desde la memoria caché local reduciendo la latencia a < 0.2 { segundos} y eliminando el costo de tokens de la API.
2. **Migración a Telemetría Distribuida (OpenTelemetry + Langfuse):** Reemplazar el archivo de texto local “.log” por un estándar de telemetría en la nube. Esto permitirá trazar de manera granular la cadena interna de pensamientos (*traces*) del agente ReAct, identificando de forma visual qué herramienta del agente consume la mayor fracción del tiempo.